

Agriclean
AN INTERNSHIP REPORT

Submitted by

Maurya Harsh

GTU Enrollment No.: 210670107091

In fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

Computer Engineering

SAL Institute of Technology and Engineering Research,

Ahmedabad



Gujarat Technological University, Ahmedabad

April, 2025



SAL Institute of Technology and Engineering Research

opposite Science City, Sola, Ahmedabad, Gujarat 380060.

CERTIFICATE

This is to certify that the internship report submitted along with the project entitled '**Agriclean**' has been carried out by **Maurya Harsh** under my guidance in fulfillment for the degree of Bachelor of Engineering in Computer Engineering, 8th Semester of Gujarat Technological University, Ahmadabad during the academic year 2024-25.

Prof. Khyati Patel
Internal Guide

Dr. Nimisha Patel
Head of CE/CSE/ICT
Department,
SALITER



Date: - 22/04/2025

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Harsh Yashkumar Maurya**, a student of SAL Institute of Technology and Engineering Research has successfully completed his/her internship in the field of MERN STACK from 20th January 2025 to 19th April 2025 12 Weeks under the guidance of Rahul Kirpekar.

His internship activities include Project Understanding & Requirement Analysis, Technical Documentation, Learning New Technologies, Development Activities, Testing & Quality Assurance, Soft Skills & Reporting.

During the internship period, he/she was exposed to various processes and was found to be diligent, hardworking, and inquisitive.

We wish him/her all the best in future endeavors.

For, Grownited Private Limited.

Rahul Kirpekar

(Authorised Signature)

301-305, 3rd Floor, Surbhi Complex, Nr. Municipal Market, Chimanlal Girdharlal Road, Navrangpura, Ahmedabad Gujarat- 380009, 7874014621, hr@grownited.com

SAL Institute of Technology and Engineering Research

opposite Science City, Sola, Ahmedabad, Gujarat 380060.

DECLARATION

I hereby declare that the Internship report submitted along with the Project entitled **Agriclean** submitted in fulfillment for the degree of Bachelor of Engineering in Computer Engineering to Gujarat Technological University, Ahmedabad, is a Bonafide record of original project work carried out by me at **Grownited Private Limited** . under the supervision of **Rahul Kirpekar** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

Name of the Student

Maurya Harsh

Sign of Student

ACKNOWLEDGEMENT

The completion of any project brings with it a sense of satisfaction, but it is never complete without thanking those individuals who made it possible and whose constant support has crowned our efforts with triumph. One cannot even imagine the power of the force that guides us all and neither can we succeed without acknowledging it.

I would like to express my gratitude to Dr. Nimisha Patel, Head of the Department of CE/CSE/ICT, SAL Institute of Technology and Engineering Research, Ahmedabad, for encouraging and inspiring me to carry out the project. I would also like to thank my internal guide, Prof. Khyati Patel, for her expert guidance, encouragement and valuable suggestions at every step of the project. I would also like to thank all the staff members of the institute for providing me with the required facilities and support towards the completion of the project.

I would also like to take this opportunity to thank my external guide, Rahul kirpekar of Grownited Private Limited, for mentoring me in the entire internship and providing me with the required resources and guidance during the project.

ABSTRACT

Agriclean is a web-based waste-management platform designed to connect farmers and certified processors within a unified, role-driven ecosystem. Processors can self-register, detail their processing capabilities, and set pricing for different waste types. Once approved, they gain access to a dedicated dashboard where incoming service requests appear in real time, simplifying the onboarding and listing process.

On the farmer side, users browse a catalog of registered processors filtered by waste category and price, then submit booking orders directly through the interface. Each order flows seamlessly into the processor's dashboard, where its status can be updated from "Collected" to "Processed." This bidirectional, real-time communication ensures transparency throughout the waste lifecycle and minimizes the need for manual follow-ups or phone calls.

Administrators oversee platform integrity by validating processor credentials, managing farmer and processor accounts, and monitoring overall activity through a centralized console. By automating matchmaking, booking, and status tracking, Agriclean reduces administrative overhead, optimizes resource utilization, and fosters environmentally responsible disposal practices. Its modular architecture not only highlights core competencies in web development and real-time data handling but also provides a scalable foundation for future features such as analytics dashboards, dynamic pricing algorithms, and geospatial routing for optimized waste collection.

List of figures

Fig 2.1 Android Development	3
Fig 2.2 Web Development	4
Fig 3.1 Process of internship project	12
Fig 3.2 Agile Methodology	13
Fig 3.3 Gantt Chart	18
Fig 4.1 Flow process in system	23
Fig 5.1 Use case diagram	27
Fig 5.2 Class diagram	28
Fig 5.3 Activity diagram	29
Fig 5.4 DFD	31
Fig 5.5 Sequence diagram	32
Fig 5.6 ER-Diagram	33
Fig 6.1 HomePage	45
Fig 6.2 Booking Page	45
Fig 6.3 Marketplace	46
Fig 6.4 Create Account	46
Fig 6.5 My Booking	47
Fig 6.6 Add Processors	48
Fig 6.7 All Bookings	48
Fig 6.8 Edit Profile	49
Fig 6.9 All Processors	49
Fig 7.1 White-box testing	50
Fig 7.2 Google performance test	59

List of Table

Table 3.1 Project/internship Effort and time	15
Table 3.2 Information domain value	16
Table 3.3 Value Adjustment Factor (VAF)	17
Table 3.4 Internship activity	19
Table 5.1 user data dictionary	30
Table 5.2 server data dictionary	30
Table 7.1 testing result	43

ABBREVIATIONS

1. HTML – Hyper Text Markup Language
2. CSS – Cascading Style Sheet
3. JS – JavaScript
4. API – Application Programming Interfacing
5. Ui- User Interface
6. UX – User Experience
7. JWT – JSON Web Token
8. DOM- Document Object Model
9. DB- Data Base
10. CRUD – Create, Read, Update, Delete
11. MERN – MongoDB, Express.js, React.js, Node.js
12. JSON – JavaScript Object Notation
13. ER- Entity Relationship
14. DFD- Data Flow Diagram
15. VS- Visual Studio

Table of Contents

DECLARATION	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
ABBREVIATIONS	vi
CHAPTER 1: OVERVIEW OF THE COMPANY.....	1
1.1 HISTORY	1
1.2 SCOPE OF WORK	2
CHAPTER 2: OVERVIEW OF DIFFERENT PROCESSES IN THE COMPANY .	3
2.1 DIFFERENT DEPARTMENT OF COMPANY	3
2.2 TECH-STACK FOR DIFFERENT DEPARTMENT.....	4
CHAPTER 3: INTRODUCTION TO INTERNSHIP REPORT	8
3.1 INTERNSHIP SUMMARY	8
3.2 PURPOSE.....	8
3.3 OBJECTIVE OF THE PROJECT	8
3.4 SCOPE.....	9
3.5 TECHNOLOGY & LITERATURE REVIEW.....	9
3.5.1 Technology Review	9
3.6 INTERNSHIP PLANNING	12
3.6.1 Project Development Approach	12
3.6.2 Internship phases.....	13
3.6.3 Internship Effort and Time, Cost Estimation	15
3.6.4 Cost Analysis	17
3.7 INTERNSHIP SCHEDULING	18
CHAPTER 4: SYSTEM ANALYSIS	20
4.1 STUDY OF CURRENT SYSTEM	20
4.2 WEAKNESS OF CURRENT SYSTEM.....	20
4.3 REQUIREMENT OF NEW SYSTEM.....	20
4.4 SYSTEM FEASIBILITY	21

4.5 MAIN PROCESSES OF THE PROPOSED SYSTEM.....	22
4.6 HARDWARE AND SOFTWARE REQUIREMENT	24
CHAPTER 5: SYSTEM DESIGN	25
5.1 SYSTEM DESIGN AND METHODOLOGY	25
5.2 DATABASE DESIGN / DATA STRUCTURE DESIGN / CIRCUIT DESIGN / PROCESS DESIGN	26
5.2.1 Use case diagram	26
5.2.2 Class diagram.....	27
5.3.3 Activity Diagram.....	28
5.3.4 Data Dictionary	30
5.5.5 Data flow diagram.....	30
5.5.6 Sequence diagram	32
5.5.7 ER-Diagram	33
CHAPTER-6 IMPLEMENTATION	34
6.1 IMPLEMENTATION ENVIRONMENT	34
6.2 MODULE SPECIFICATION.....	34
6.3 RESULTS.....	35
6.4 CODING STANDARD.....	39
CHAPTER-7 TESTING	40
7.1 TEST PLANNING	40
7.2 TESTING METHODS	41
7.3 TEST CASES	42
CHAPTER-8 CONCLUSION AND DISCUSSION	44
8.1 OVERALL ANALYSIS OF INTERNSHIP / PROJECT VIABILITIES	44
8.2 SUMMERY OF INTERNSHIP	44
8.3 LIMITATION AND FUTURE ENHANCEMENT.....	45
8.3.1 Limitation.....	43
8.3.2 Future enhancement	43
8.4 CONCLUSION	45
CHAPTER-9 REFERENCE.....	46

CHAPTER 1: OVERVIEW OF THE COMPANY

1.1 HISTORY

Grownited Private Limited is a fast-growing software development company located in Ahmedabad, Gujarat. Founded with the vision to create technology that solves real-world problems, Grownited began its journey with a small but skilled team of developers and innovators focused on delivering scalable and innovative digital solutions for businesses.

The company initially focused on basic web development services but quickly evolved by embracing modern frameworks and full-stack technologies. Over time, Grownited expanded its services into mobile app development, cloud integration, UI/UX design, and enterprise-level platforms. Their expertise in the MERN (MongoDB, Express.js, React.js, Node.js) stack allowed them to offer powerful and dynamic solutions for clients in sectors like healthcare, education, and logistics.

What set Grownited apart was its client-centric approach and commitment to quality. By adopting agile methodologies, the company ensured continuous improvement and faster delivery cycles. Every project is handled with a focus on innovation, performance, and usability, resulting in solutions that are both technically sound and user-friendly.

In addition to commercial work, Grownited has been invested in education and training. The company offers internships and industrial training programs to computer engineering students and aspiring developers, providing hands-on experience in real projects and modern tools.

Today, Grownited continues to grow its presence in the Indian tech ecosystem. With a forward-thinking mindset and a strong dedication to excellence, the company is not only delivering software products but also nurturing the next generation of tech talent—playing a key role in shaping the digital future of the country.

1.2 SCOPE OF WORK

At Grownited Private Limited, we provide a broad range of services designed to meet the unique needs of our clients. Our core offerings include:

Software Development: We specialize in creating custom software solutions tailored to streamline business operations, enhance productivity, and foster growth. From initial consultation to final deployment, we ensure that our solutions meet the specific needs of our clients.

Mobile Application Development: Our team develops user-friendly, secure, and scalable mobile applications for both Android and iOS platforms. We leverage the latest technologies to create apps that improve user experience and help businesses engage their customers effectively.

Web Development: We offer comprehensive web development services, including designing and building dynamic websites and web applications. Our solutions are secure, scalable, and optimized to meet the specific requirements of each client, whether it's an e-commerce platform or a corporate website.

Consulting Services: We provide expert consulting to help organizations align their technology and strategies with their business goals. Our services include digital transformation, process optimization, and IT strategy guidance to ensure clients remain competitive in a fast-evolving market.

UI/UX Design: We focus on creating intuitive and visually stunning designs that enhance user engagement and improve the overall user experience. Our UI/UX team ensures that every product we deliver is user-centered and aesthetically pleasing.

CHAPTER 2: OVERVIEW OF DIFFERENT PROCESSES IN THE COMPANY

2.1 DIFFERENT DEPARTMENT OF COMPANY

- **Application Development**

The Application Development Department at Grownited Private Limited is responsible for creating custom software applications tailored to client-specific requirements. This process involves multiple stages, starting from conceptualization, where developers and clients collaborate to define the application's objectives, scope, and features. The design team then creates wireframes and UI/UX mockups to visualize the application's interface and flow. During the development phase, coding is done using modern programming languages and frameworks, ensuring functionality, performance, and scalability. The applications undergo rigorous testing to detect and eliminate bugs, making sure they function smoothly across devices and platforms. Once finalized, the applications are deployed to production environments, with the department also offering post-deployment support, updates, and performance monitoring. Grownited develops applications across multiple platforms, including Android apps, iOS apps, and Desktop applications, ensuring comprehensive coverage for all user bases.



Fig 2.1 Android development

- **Web Application**

The **Web Development Department** at Grownited is dedicated to building responsive, high-performance websites and web applications that are accessible across all modern browsers and devices. This process starts with the design phase, where the layout, color schemes, and user interaction flow are planned. Designers use tools like Figma, Adobe XD, and Photoshop to craft visually compelling mockups. The front-end development team then brings these designs to life using technologies like **HTML**, **CSS**, **JavaScript**, and popular libraries such as **React.js**. Meanwhile, the back-end team manages server-side logic, APIs, and databases using technologies like **Node.js**, **PHP**, or **Python** frameworks. In addition, web server configuration and deployment are handled to ensure robust performance, security, and uptime. The final product is a secure, scalable, and visually engaging web platform, optimized for speed and usability, tailored to client goals and end-user expectations.



Fig 2.2 Web development

2.2 TECH-STACK FOR DIFFERENT DEPARTMENT

Application Development

- **Deployment platform and Programing language:**

Agriclean is a cross-platform, full-stack agricultural waste management application designed to connect farmers, processors, and administrators through a centralized digital platform. The system is primarily web-based and supports mobile responsiveness for ease of use in rural environments. Below is the structure of the application in terms of platform support and development technologies:

1. **Web Application (Primary Platform):** Agriclean is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js), ensuring efficient data processing, real-time interaction, and scalability across all major browsers.
 - **Frontend:** Built using **React.js**, a JavaScript library for building modular and dynamic user interfaces, enabling seamless role-based dashboards for farmers, processors, and admins.
 - **Backend:** Powered by **Node.js** and **Express.js**, which handle all API communications, routing logic, and middleware for core functionalities like bookings and status updates.
 - **Database:** Utilizes **MongoDB**, a flexible NoSQL database ideal for managing structured and semi-structured data related to users, waste types, and bookings.
2. **Mobile Support (Responsive Web App):** Though a native mobile application has not been developed yet, Agriclean is fully responsive and optimized for mobile browsers, allowing farmers and processors to access the platform conveniently from smartphones and tablets.
3. **Cross-Platform Potential(Planned feature):** For future scalability, Agriclean is designed with a flexible architecture to support cross-platform development:
 - **React Native (Future Implementation):** Since the frontend uses React.js, it can easily be transitioned into **React Native** to build native Android and iOS applications using the same codebase, ensuring consistent performance across platforms.
4. **Cross-Platform Development:** Cross-platform development allows developers to build apps that can run on multiple platforms with a single codebase. Some popular cross-platform development frameworks include:

Flutter: Developed by Google, Flutter is an open-source UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase. It uses the Dart programming language.

React Native: Created by Facebook, React Native is a JavaScript framework for building mobile applications using React. It enables developers to write code once and deploy it across multiple platforms, including iOS and Android.

Xamarin: Developed by Microsoft, Xamarin is a cross-platform development framework that allows developers to build native iOS, Android, and Windows apps using C# and .NET.

Web Development

Frontend:

- **HTML:** HTML, or Hypertext Markup Language, is the backbone of web development. It uses tags to structure content on web pages, defining elements like headings, paragraphs, links, images, and forms. HTML provides the foundation for creating structured and semantically meaningful content that web browsers can understand and render.
- **CSS:** CSS, or Cascading Style Sheets, is a language used to style HTML documents, defining the layout, colors, and other visual aspects of web pages. It allows developers to create visually appealing designs and ensure consistency across different devices. Tailwind CSS, on the other hand, is a utility-first CSS framework that provides pre-designed utility classes to streamline the process of building user interfaces.
- **Bootstrap:** Bootstrap is a popular front-end framework for building responsive and mobile-first websites and web applications. Developed by Twitter, it provides a collection of HTML, CSS, and JavaScript components and utilities that simplify the process of designing and styling web pages. Bootstrap includes a grid system for laying out content, pre-styled components such as buttons, forms, and navigation bars, as well as responsive design classes to ensure compatibility with various screen sizes and devices.

React.JS: React.js, commonly referred to as React, is a JavaScript library for building user interfaces. Developed by Facebook, React enables developers to create interactive and dynamic UI components that can efficiently update in

response to user interactions and data changes. React utilizes a component-based architecture, allowing developers to break down complex user interfaces into reusable and modular components.

Backend:

- **Express.js:** Express.js is a lightweight and flexible **backend web framework for Node.js** used to build the server-side of the application. It handles routing, API requests, middleware, and server-side logic for features such as user registration, login authentication, vaccine booking, and hospital management.
- **Node.js:** Node.js serves as the **runtime environment** for executing backend JavaScript code. It allows VaxTrack to handle multiple API requests asynchronously, manage secure data transmission, and communicate with the MongoDB database efficiently.

Database:

- **MongoDB:** MongoDB is a popular open-source NoSQL database management system known for its flexibility, scalability, and ease of use. Unlike traditional relational databases, MongoDB stores data in flexible, JSON-like documents called BSON (Binary JSON), allowing for a dynamic and schema-less data model. This makes MongoDB well-suited for handling unstructured or semi-structured data and accommodating evolving data schemas.

CHAPTER 3: INTRODUCTION TO INTERNSHIP REPORT

3.1 INTERNSHIP SUMMARY

The goal of the 'Agriclean' internship project is to develop a web-based waste management system that facilitates smooth coordination between farmers, waste processors, and administrators on a single digital platform. The application allows processors to register, add waste types they handle, and set the prices they offer for waste collection. Farmers can view these processors, compare offerings, and book waste pickup based on their needs. The processor can then update the status of the booking to “Collected” and later to “Processed.” Admins oversee the entire system, managing processor and farmer data, verifying information, and monitoring all booking activity via a central dashboard. Each user role is provided with a dedicated interface designed for their specific actions and workflows. The platform ensures real-time reflection of bookings, status updates, and system data, delivering a seamless experience across all devices. This project emphasizes a responsive and intuitive user interface with secure, role-based access for clear and organized functionality.

Platform Used: HTML, Tailwind CSS, JavaScript, React.js, Node.js, Express.js, MongoDB

3.2 PURPOSE

The purpose of Agriclean is to streamline the agricultural waste management process by connecting farmers, processors, and administrators on a unified platform. It facilitates processor registration, waste booking, status updates, and user data management in real time, enhancing operational transparency and simplifying coordination using modern web technologies.

Goals of the Project:

- Provide an easy-to-use interface for farmers, processors, and admins.
- Enable processors to list services and manage booking statuses.
- Allow farmers to browse and book waste processing services.
- Offer admins tools to manage users and monitor system activity.
- Ensure real-time synchronization of bookings and updates.
- Promote sustainable waste management practices through digitization.

3.3 OBJECTIVE OF THE PROJECT

The objective of the Agriclean project is to develop an efficient, web-based system that digitizes the agricultural waste collection and processing workflow. The platform aims to empower processors with tools to manage waste collection and pricing, support farmers in booking suitable processors based on waste type, and equip administrators with full control over platform data and activity—all while ensuring seamless, real-time communication and user experience.

3.4 SCOPE

What can it do?

- Processors can register and list waste processing details and prices.
- Farmers can browse processors and book waste collection services.
- Processors can update the booking status to "Collected" or "Processed."
- Admins can manage processor and farmer accounts and monitor bookings.
- Users can interact via dedicated dashboards based on their roles.
- The system provides real-time updates and mobile-responsive access.

3.5 TECHNOLOGY & LITERATURE REVIEW

3.5.1 Technology Review

- **MongoDB**

MongoDB is a widely-used NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). It's designed to handle large-scale, dynamic data and is particularly well-suited for applications requiring rapid data access. Here's a brief technology review highlighting its key aspects:

1. **Flexible Schema:** MongoDB offers a flexible, schema-less design, allowing developers to store documents with varying structures. This is useful in applications like VaxTrack, where data might vary between patients, hospitals, or vaccine types. The flexibility allows for the easy addition of new fields as the application evolves.
2. **Horizontal Scalability:** MongoDB is built for horizontal scalability, meaning it can distribute data across multiple servers or clusters. This enables VaxTrack to handle increasing amounts of data and traffic efficiently, ensuring the system can scale as more hospitals, patients, and vaccines are added.
3. **Real-Time Data Processing:** MongoDB's real-time data synchronization is beneficial for applications that require frequent updates, such as VaxTrack. For instance, when a vaccine is booked, the system can immediately reflect the new data across all users and devices.
4. **Aggregation Framework:** MongoDB provides powerful aggregation capabilities that allow developers to process and analyze large sets of data directly within the database. This feature can be used in VaxTrack to generate reports, track vaccine inventory, and analyze user interactions.
5. **Replication and High Availability:** MongoDB supports automatic replication, which means data is copied across multiple servers to ensure high availability and disaster recovery. This feature is critical for maintaining uninterrupted service in VaxTrack, even during server outages or high traffic periods.

- **React.js**

React.js, a popular JavaScript library developed by Facebook, has become a cornerstone technology for building dynamic and interactive user interfaces. Here's a brief technology review highlighting its key aspects:

1. **Component-Based Architecture:** React.js follows a component-based architecture, allowing developers to break down UI elements into reusable components. This modular approach enhances code maintainability, scalability, and reusability.
2. **Virtual DOM:** React utilizes a virtual DOM (Document Object Model) to efficiently update the user interface. Instead of directly manipulating the DOM, React compares the virtual DOM with the actual DOM, identifying only the necessary updates and minimizing rendering overhead. This results in improved performance and responsiveness of web applications.
3. **JSX (JavaScript XML):** JSX is a syntax extension for JavaScript used in React to describe the UI components. It enables developers to write HTML-like code directly within JavaScript, making the code more readable and expressive. JSX is compiled into regular JavaScript functions by tools like Babel before being executed in the browser.
4. **Unidirectional Data Flow:** React follows a unidirectional data flow paradigm, where data flows only in one direction, typically from parent components to child components. This ensures predictable data flow and facilitates easier debugging and maintenance of applications.
5. **State Management:** React allows the management of component state, which represents the data specific to a component and determines its behavior and appearance. While React has its built-in state management capabilities, developers often use external libraries like Redux or Context API for managing complex application states.

- **Node.js**

Node.js is a JavaScript runtime environment that allows developers to execute JavaScript code on the server side. It has gained significant popularity for building scalable and efficient server-side applications. Here's a brief technology review highlighting its key aspects:

1. **Non-Blocking I/O Model:** Node.js uses an event-driven, non-blocking I/O model, which means it can handle multiple requests simultaneously without waiting for previous ones to complete. This results in improved performance and responsiveness, especially for applications requiring real-time data processing.
2. **Single Language Stack:** With Node.js, developers can use JavaScript for both front-end and back-end development, making the development process smoother and reducing the learning curve. This unified stack increases development efficiency and streamlines collaboration between developers.
3. **Scalability:** Node.js is designed for building scalable applications. Its event-driven architecture and asynchronous nature make it ideal for applications with high traffic, such as messaging platforms or real-time systems like **VaxTrack**, which require concurrent processing and low-latency performance.
4. **NPM (Node Package Manager):** Node.js benefits from a vast ecosystem of libraries and modules available through NPM. This allows developers to easily integrate third-party packages, speeding up development time and expanding functionality without reinventing the wheel.
5. **Real-Time Data Processing:** Node.js is well-suited for building real-time applications such as chat systems or online gaming platforms. In the context of **VaxTrack**, Node.js allows the platform to handle real-time updates of vaccine bookings, hospital stock, and patient status with minimal latency.

- **Express.JS**

Express.js, often referred to as Express, is a minimalist web application framework for Node.js. It provides a robust set of features for building web and mobile applications, APIs, and server-side applications. Here's a technology review highlighting key aspects of Express.js:

1. **Minimalist and Lightweight:** Express.js is designed to be lightweight and unopinionated, providing developers with the flexibility to build applications according to their preferences. It offers a minimalistic core that can be extended with various middleware and libraries as needed.
2. **Middleware:** Middleware functions in Express.js are the heart of its functionality. They are functions that have access to the request and response objects and can perform tasks such as parsing request bodies, handling sessions, authentication, logging, and error handling. Middleware functions can be chained together to create powerful request processing pipelines.
3. **Routing:** Express provides a simple yet powerful routing mechanism that allows developers to define routes for different HTTP methods and URLs. Routes can be defined with parameters and can handle dynamic URL segments. This enables developers to create RESTful APIs and handle complex request-response logic effectively.
4. **Template Engines:** Although Express.js itself does not include a template engine, it allows developers to integrate various template engines such as Pug (formerly Jade), EJS, Handlebars, and Mustache. These template engines enable server-side rendering of HTML pages, facilitating the creation of dynamic and data-driven web applications.

3.6 INTERNSHIP PLANNING

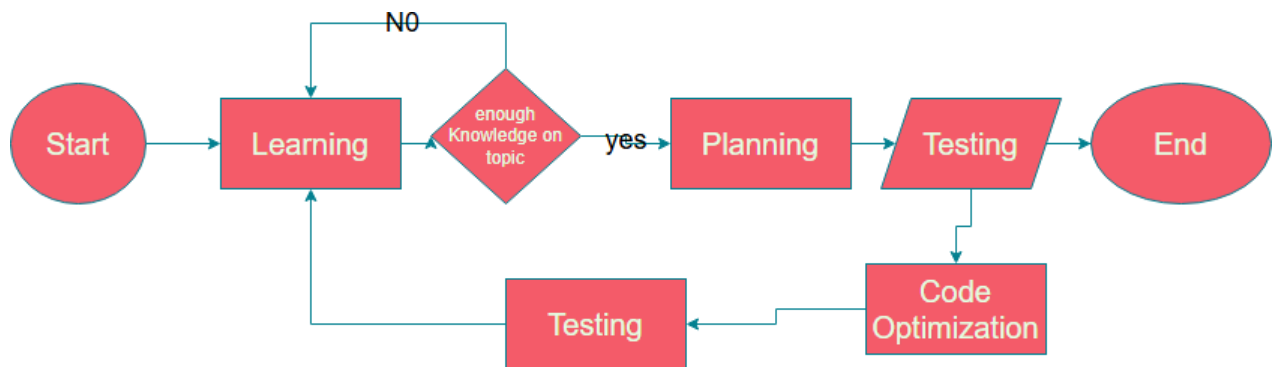


Fig 3.1 process of internship project

3.6.1 Project Development Approach

A project development approach refers to the overarching strategy or methodology used to plan, execute, and manage the development of a project from inception to completion. There are several project development approaches, each with its own principles, processes, and practices.

Agile methodology

The Agile methodology is an iterative and incremental approach to software development that prioritizes flexibility, collaboration, and customer feedback. It emerged as a response to the limitations of traditional waterfall methodologies, which often resulted in lengthy development cycles, rigid processes, and difficulty adapting to changing requirements. here are the steps involves in agile methodology:

1. Project Initiation and Planning:
 - Define project vision, goals, and initial requirements.
 - Create a prioritized product backlog.
2. Sprint Planning:
 - Select features from the backlog for the upcoming sprint.
 - Break features into smaller tasks and estimate effort.
 - Set sprint goals and acceptance criteria.
3. Sprint Execution:

- Develop and implement selected features and tasks.
 - Collaborate closely and communicate progress daily.
 - Continuously test and integrate new features.
4. Sprint Review and Retrospective:
- Demonstrate completed work to stakeholders and gather feedback.
 - Review progress towards sprint goals.
 - Reflect on sprint process, identify improvements, and plan adjustments.
5. Iterative Development and Continuous Improvement:
- Repeat the sprint cycle, adapting plans based on feedback.
 - Prioritize and refine the product backlog continuously.
 - Strive for continuous improvement in processes, product quality, and team efficiency.

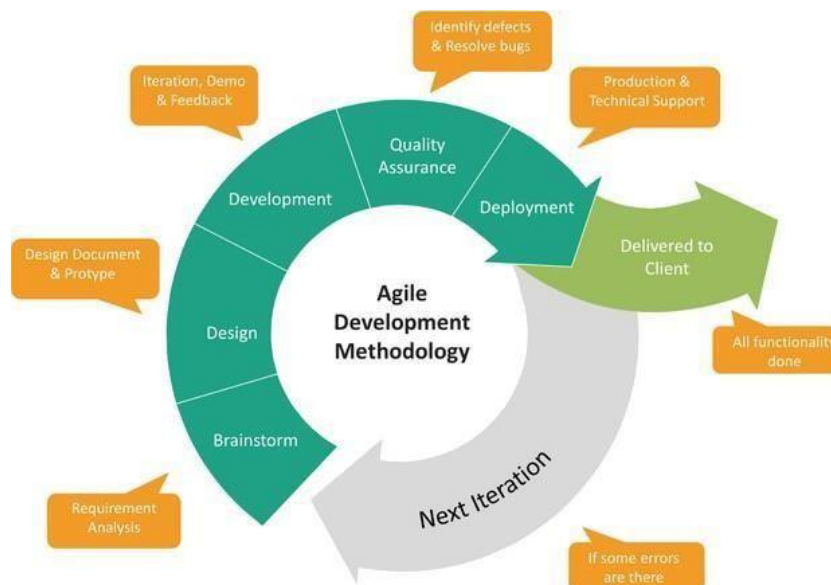


Fig 3.2 Agile Methodology

3.6.2 Internship phases:

1. Training:

During the training sessions, the initial focus was on acquiring fundamental skills in HTML and CSS, laying the groundwork for understanding web development principles and building basic user interfaces. With these foundational concepts in place, the training

progressed to learning JavaScript, a powerful scripting language essential for creating dynamic and interactive web content. Through practical exercises and tasks assigned by the mentor, proficiency in JavaScript was developed, enabling the manipulation of web page elements and the implementation of more advanced functionalities. Subsequently, as the grasp of HTML, CSS, and JavaScript solidified, the training transitioned to learning React.js, a popular JavaScript library for building user interfaces. This phase involved studying React's component-based architecture, state management, and virtual DOM manipulation, while also applying acquired skills to develop more complex and interactive web applications. Overall, the training journey encompassed a gradual progression from foundational concepts to more advanced technologies, providing a comprehensive understanding of web development principles and tools.

2. Implementing training into project:

Utilizing the knowledge gained during training sessions, I seamlessly integrated foundational web development concepts into my internship project, a "chat room" application. Beginning with HTML and CSS, I crafted the basic structure and visual layout of the chat room interface, ensuring a user-friendly design and intuitive navigation. Leveraging JavaScript, I implemented interactive features such as real-time messaging, enhancing the overall functionality and user experience of the application. Furthermore, by incorporating React.js, I adopted a component-based approach to streamline development, facilitating the creation of reusable UI elements and optimizing performance. This internship project served as a practical application of my training, allowing me to demonstrate proficiency in web development and showcase my ability to translate theoretical knowledge into real-world projects. Throughout the process, I continuously refined and expanded upon my skills, further solidifying my understanding of web technologies and their practical applications.

3.6.3 Internship Effort and Time, Cost Estimation

4

Activity	Estimated Duration
Project Initiation	1 week
Initial Proposal	2 weeks
Building a Project Plan	1 week
Literature Review	5 days
Methodology	1 week
System Analysis	1 weeks
Project Design	1 week and 6 days
Project Development	6 days
Testing	6 days
Critical Analysis	4 days
Evaluation	1 day
Report Writing	1 week
Total Effort	12 weeks (Approx. 3 months)

Table 3.1 Project/Internship Effort and Time

Effort Estimation

To ensure a proper evaluation of the Agriclean system and to estimate the time and effort required for its development, metrics-based calculations were applied. Software project metrics provide a structured way to understand the size, complexity, and scope of a project, enabling accurate planning and cost estimation. For the Agriclean platform, **function-oriented metrics** were used to determine the size of the system at an abstract level, independent of the programming language used. One of the most effective approaches for this is the **Function Point (FP)** method.

- Number of external inputs (EIs) –user-generated or system-generated inputs.
- Number of external outputs (EOs) –outputs that the system processes or sends to other systems.
- Number of external inquiries (EQs) – user inputs that result in immediate output.
- Number of internal logical files (ILFs) – user inputs that result in immediate output.
- Number of external interfaces files (EIFs) - data files used by the application but maintained by other systems.

Step 1: You must compute the count-total which will be used to define the complexity of a project.

You will do that by completing the table below:

Effort Estimation of My Project based on FP:

Measurement parameter	Count	Weighting factor	Result
Number of user inputs	5	4	20
Number of user outputs	4	5	20
Number of user inquiries	3	3	9
Number of files	2	7	14
Number of external interfaces	2	5	10
Count total=	73		

Table 3.2 Information Domain Values

Step 2: You must find the complexity adjustment values based on responses to the questions below:

1.	Does the system require reliable backup and recovery?	4
2.	Are data communications required?	4
3.	Are there distributed processing functions?	2
4.	Is performance critical?	3
5.	Will the system run in an existing, heavily utilized operational environment?	4
6.	Does the system require on-line data entry?	5
7.	Does the on-line data entry require the input transaction to be built over multiple screens or operations?	2

8.	Are the master files updated on-line?	3
9.	Are the inputs, outputs, files, or inquiries complex?	2
10.	Is the internal processing complex?	3
11.	Is the code designed to be reusable?	4
12.	Are conversion and installation included in the design?	2
13.	Is the system designed for multiple installations in different organizations?	3
14.	Is the application designed to facilitate change and ease of use by the user?	5
Count		50

Table 3.3 Value Adjustment Factors (VAF)

$$FP = \text{Count} * [0.65 + 0.01 * \text{SUM}$$

$$[fi]] FP = \text{Count} * [0.65 + 0.01 * 50]$$

$$FP = 73 * 0.92$$

$$FP = 63.16$$

For this project our FP/pm is 27.98. Hence, we need 3 months to complete the project.

4.1.2 Cost Analysis

The cost spent in the making of the project is categorized into two parts:

Direct cost (This is in terms of money):

In our project it is the estimated cost of:

Hardware (Computer) Rs14,600.00 (fixed forever)

- 8 GB RAM
- 1.5 GHZ PROCESSOR
- 15 GB FREE SPACE IN HARD DISK
- 1080*768 display

Software Configuration:

- Operating System: Windows 11, Linux and Mac

- Coding Language: JavaScript
- Tools: vs code, web browser (google chrome)

Indirect cost:

In my project it is the estimated cost in terms of:

- Time spent in system analysis and design
- Managing time for coding.
- Generating Report
- Referring other sources like the internet

4.2 INTERNSHIP SCHEDULING

• Gantt Chart

A Gantt chart is a visual representation of a project schedule, displaying tasks as horizontal bars along a timeline. Each bar represents a task's start and end dates, with its length indicating the task's duration. Dependencies between tasks are shown by their relative positioning. Milestones are marked as significant points. Gantt charts help in planning, scheduling, and tracking project progress, aiding project managers in efficiently managing resources and communicating timelines to stakeholders.

	2025-01				2025-02				2025-03				2025-04			
	13	19	26	02	09	16	23	02	09	16	23	23	30	06	13	19
Requirement & Planning	Requirements & Development															
Analysis			Analysis													
Designing					Designing											
Coding & Development								Coding & Development								
Testing															Testing	
Documentation	Documentation															

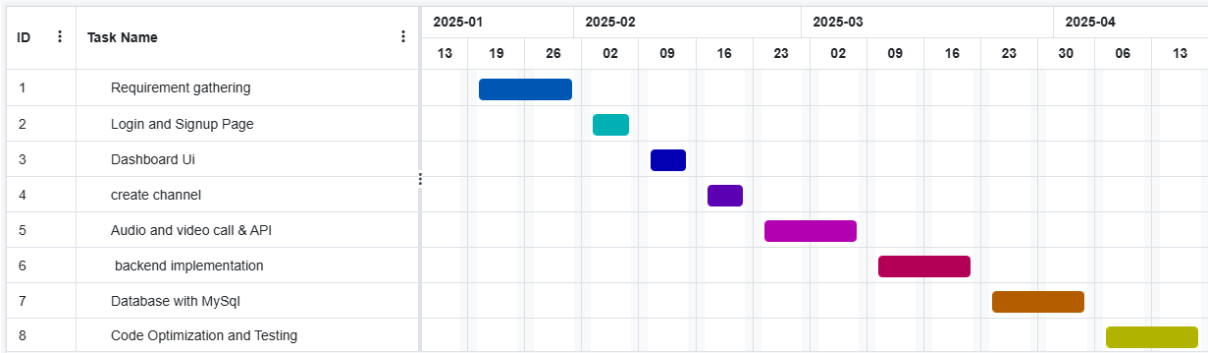


Fig 3.3 Gantt Chart

- Weekly internship activity

Week No.	Brief Information regarding Internship Activities
Week 01.	Initialized GitHub repository, set up the MERN stack (MongoDB, Express.js, React.js, Node.js), and created the base folder structure. Designed initial wireframes and system flow diagrams for Admin, Farmer, and Processor panels.
Week 02.	Developed user registration, login, and logout functionalities with role-based authentication using JWT. Ensured secure password storage with bcrypt and implemented protected routes for each role.
Week 03.	Created dashboards for all three roles (Admin, Farmer, Processor) with personalized navigation. Implemented basic routing using React Router and tested session handling.
Week 04.	Built MongoDB schemas and Express routes for processor registration and waste management details. Connected backend with the database and tested data insertion using Postman.
Week 05.	Implemented update and delete functionality for processor and waste records. Created dynamic forms using React for real-time data entry and integrated CRUD operations.
Week 06.	Designed booking models and built API routes for booking waste pickup. Developed endpoints to create, read, and track bookings for farmers and processors.
Week 07.	Integrated booking APIs into the farmer dashboard. Enabled farmers to select waste type, processor, and confirm booking. Reflected real-time booking data in the processor panel.
Week 08.	Planned geolocation-based processor recommendations using HTML5 location API (as future enhancement). Prepared backend routes and frontend logic for potential location features.
Week 09.	Added admin controls to manage all bookings and user accounts. Developed analytics using Chart.js to visualize booking trends, processor activity, and waste processing stats.
Week 10.	Styled the entire application using CSS and Tailwind CSS. Improved layout responsiveness for mobile and desktop, added icons, and introduced user-friendly design elements.
Week 11.	Performed full system testing for each module. Created test cases for all routes and UI elements. Debugged issues, improved error handling, and validated forms across all user roles.
Week 12.	Finalized documentation with screenshots, user instructions, and report formatting. Prepared presentation slides and reviewed the complete Agriclean platform for final evaluation.

Table 3.4 Internship activity

CHAPTER 4: SYSTEM ANALYSIS

4.1 STUDY OF CURRENT SYSTEM

- The traditional agricultural waste management system is largely manual and unorganized. Farmers often rely on local contacts or word-of-mouth to find processors, and bookings are typically handled via phone calls or in person. There is no centralized system to track processor availability, waste collection status, or historical records. This leads to delays, missed collections, and inefficient resource utilization. Furthermore, processors have no standardized method for advertising their services or managing collection requests, and admins have no real-time visibility over ongoing activities. The lack of a digital system results in miscommunication, poor record-keeping, and limited operational transparency.

4.2 WEAKNESS OF CURRENT SYSTEM

- Manual coordination often results in errors such as missed bookings, duplicated requests, or miscommunication between farmers and processors. There are no real-time updates regarding booking confirmations or the status of waste processing. Farmers have no reliable means of finding nearby processors or comparing service pricing. Additionally, processors lack tools for managing and tracking incoming requests efficiently. The absence of a centralized platform also prevents admins from monitoring user activity and maintaining system integrity. Poor accessibility and outdated workflows make it difficult to scale waste management efforts in rural and semi-urban areas.

4.3 REQUIREMENT OF NEW SYSTEM

- Agriclean is designed to digitize and streamline agricultural waste management by offering a centralized, web-based platform for farmers, processors, and admins. Farmers can browse registered processors, view the types of waste they accept and the prices offered, and book services instantly. Processors can manage their profiles, list services, and update booking statuses in real-time. Admins have full access to system data, enabling them to manage users and oversee all activity. The platform supports role-based login, responsive design, and dashboard views tailored to each user. Real-time updates, structured workflows, and mobile accessibility are key to improving efficiency, accuracy, and scalability in agricultural waste handling.

4.4 SYSTEM FEASIBILITY

The feasibility of Agriclean is evaluated across several dimensions, ensuring that the system can be successfully implemented and maintained within the scope of technical and operational capabilities.

- **Technical Feasibility:** The platform is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js), providing a robust and scalable architecture. React enables a dynamic and responsive UI for different user roles, while Express and Node manage backend logic and API communication. MongoDB efficiently handles structured and semi-structured data like user profiles, booking records, and service listings. Geolocation integration (future enhancement) is considered for locating nearby processors. The system supports real-time data updates and includes modules for secure login and session handling.
- **Economic Feasibility:** The use of open-source technologies minimizes licensing and development costs. Hosting can be managed through platforms like Vercel or Render, which support fast deployments and reduce infrastructure expenses. The primary costs involve initial development, server hosting, and optional third-party tools for notifications or maps.
- **Operational Feasibility:** Agriclean is user-friendly, requiring minimal training for farmers, processors, and admins. The booking system is intuitive and mobile-responsive, making it accessible even in low-tech environments. Admins benefit from an organized backend for managing all users and activities. Automating booking, status updates, and processor listings improves day-to-day operations and reduces manual intervention.
- **Legal and Regulatory Feasibility:** The platform follows best practices for data privacy and security. Sensitive user data is encrypted and stored securely, and all user actions are based on consent and terms of service. The system is designed to comply with digital service regulations and privacy norms relevant to user account management and transactional data.
- **Scheduling Feasibility:** The development of Agriclean was planned over a 12-week internship, with milestones covering UI design, backend integration, database

modeling, and testing. Regular iterations and code reviews ensured progress remained on track, leading to a timely and functional product deployment.

4.5 MAIN PROCESSES OF THE PROPOSED SYSTEM

The major processes of the new VaxTrack system are as follows:

- **User Sign-Up/Login:** Farmers, processors, and admins access the platform through separate, role-based login panels. New users can register with their details, while existing users log in securely to access their respective dashboards.
- **Processor Registration & Waste Listing:** Processors can register and provide detailed information about the types of agricultural waste they accept and the price they offer for collection. This data is displayed to farmers for comparison and selection.
- **Book Waste Collection:** Farmers can browse available processors based on waste type and pricing. Once a suitable processor is selected, the farmer can initiate a booking request for waste collection.
- **Status Update & Tracking:** Once a booking is confirmed, the processor can update the booking status to "**Collected**" and then to "**Processed**", allowing farmers and admins to track the progress in real time.
- **Admin Control Panel:** Admins can manage user accounts (farmers and processors), verify registration details, oversee booking history, and ensure that the data and operations on the platform remain accurate and secure.
- **Farmer Dashboard:** Farmers can view their booking history, check the status of current bookings, and review processor ratings and services for future planning.
- **Processor Dashboard:** Processors can manage incoming booking requests, update service status, and review past interactions with farmers, ensuring smooth operation and waste collection scheduling.
- **Analytics & Reports:** Admins can access dashboards that display system-wide statistics including number of bookings, active users, processed waste quantities, and processor participation, using visual tools for clear reporting and analysis.

Fig 4.1 flow of processes in system

4.6 HARDWARE AND SOFTWARE REQUIREMENT

Hardware:

- There is no specific requirement for hardware.
 - RAM
 - Storage

Software Requirements:

Client Side (Frontend):

- **Operating System:** Windows 10/11
- **Browser(for Testing):** Google Chrome, Mozilla Firefox, or Microsoft Edge (latest versions)
- **Development Tools:**
 - Visual Studio Code
 - Node.js and npm
 - Vite (Frontend build tool)
 - React.js (for Frontend)

Server Side (Backend):

- **Operating System:** Linux (Ubuntu preferred) / Windows Server
- **Runtime Environment:** Node.js (v18 or above)
- **Frameworks & Libraries:**
 - Express.js (for API and server-side logic)
 - Mongoose (for MongoDB interaction)

Database:

- **MongoDB** (Cloud or Local instance)

Other Tools:

- **Postman** (for API testing)
- **Git** (for version control)

- **MongoDB Compass** (for database visualization and management)

CHAPTER 5: SYSTEM DESIGN

5.1 SYSTEM DESIGN AND METHODOLOGY

- **Design approach:**

In designing the Agriclean web application, the process begins with in-depth research to understand user needs, platform objectives, and communication patterns. This informs the definition of clear goals, scope, and success metrics, following a user-centered design approach.

User personas, journeys, and use cases are developed to guide decision-making. Through ideation and conceptualization, various design solutions are explored and iterated upon based on user feedback. Visual and interaction design focuses on clarity, consistency, and intuitive use across different features like booking, scheduling etc.

- **System design**

System design is a foundational phase in the development of the Agriclean web application, establishing the architectural framework and defining the interaction of its components to deliver a seamless real-time communication platform. It begins with an in-depth analysis of functional requirements—such as real-time messaging, voice/video calls, and server management—and non-functional requirements including scalability, low latency, and high availability.

The system architecture is structured using a microservices-based approach, where core functionalities like authentication, messaging, media handling, and notifications operate as independent but interconnected services. This modular structure ensures flexibility, ease of maintenance, and support for large-scale concurrent usage.

Integration and communication mechanisms are carefully considered, ensuring smooth interactions between system components and external services. Throughout the design process, collaboration among stakeholders is paramount, facilitating alignment with project goals and requirements. Ultimately, a well-crafted system design serves as the foundation for the development of robust and scalable software solutions.

5.2 DATABASE DESIGN / DATA STRUCTURE DESIGN / CIRCUIT DESIGN / PROCESS DESIGN

5.2.1 Use case diagram:

A use case diagram is a graphical representation in Unified Modeling Language (UML) that depicts the interactions between actors (users or systems) and a system under consideration to achieve specific goals or functionalities. It provides a high-level overview of the system's functionalities from the perspective of its users, illustrating various use cases (functional requirements) and their relationships with actors. Actors are represented as stick figures, while use cases are depicted as ovals. Relationships between actors and use cases are shown using arrows to indicate associations, with optional inclusion and extension relationships to illustrate additional behaviors or variations within a use case.

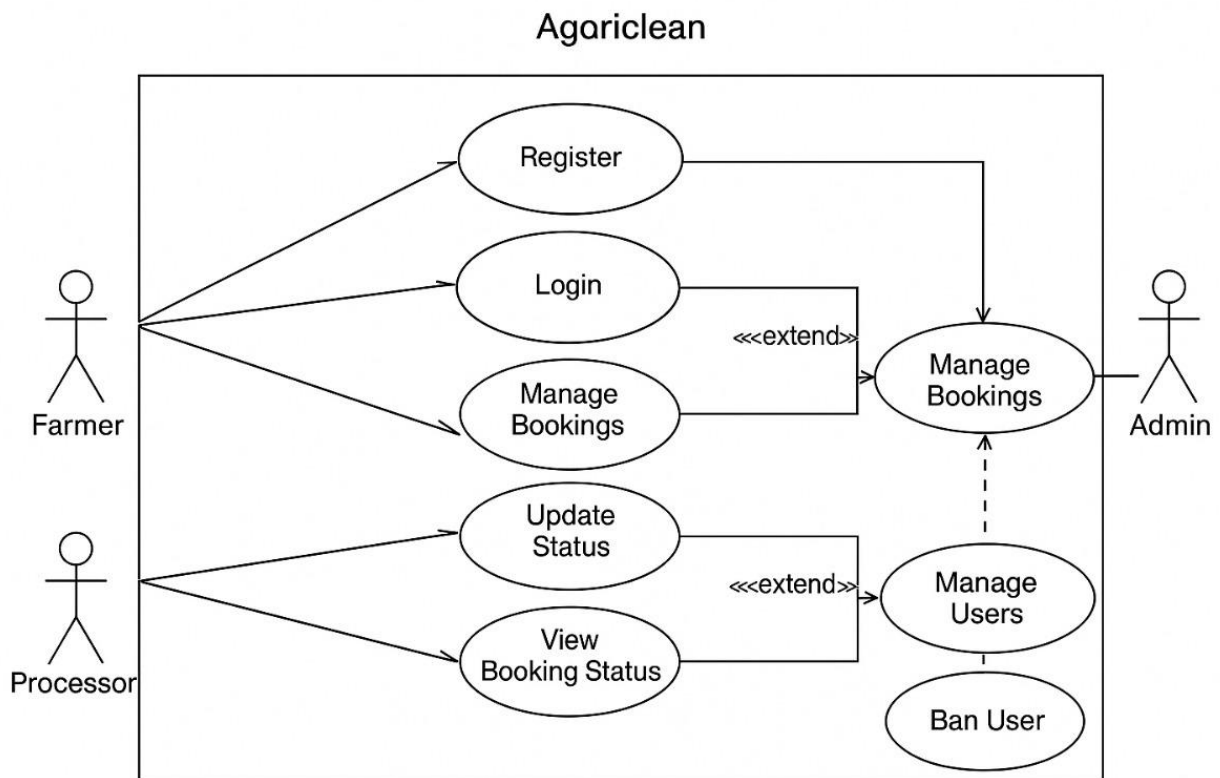


Fig 5.1 Use Case diagram

5.2.2 Class diagram:

A class diagram is a type of static structure diagram in Unified Modeling Language (UML) that represents the structure and relationships of classes within a system. It visually depicts the classes, attributes, operations, and associations between classes in a system. Classes are represented as rectangles, with the class name positioned at the top, followed by its attributes and operations listed beneath. Associations between classes are depicted as lines connecting them, indicating relationships such as associations, aggregations, or compositions. Multiplicity notations may be used to specify the cardinality of associations, indicating the number of instances of one class related to another.

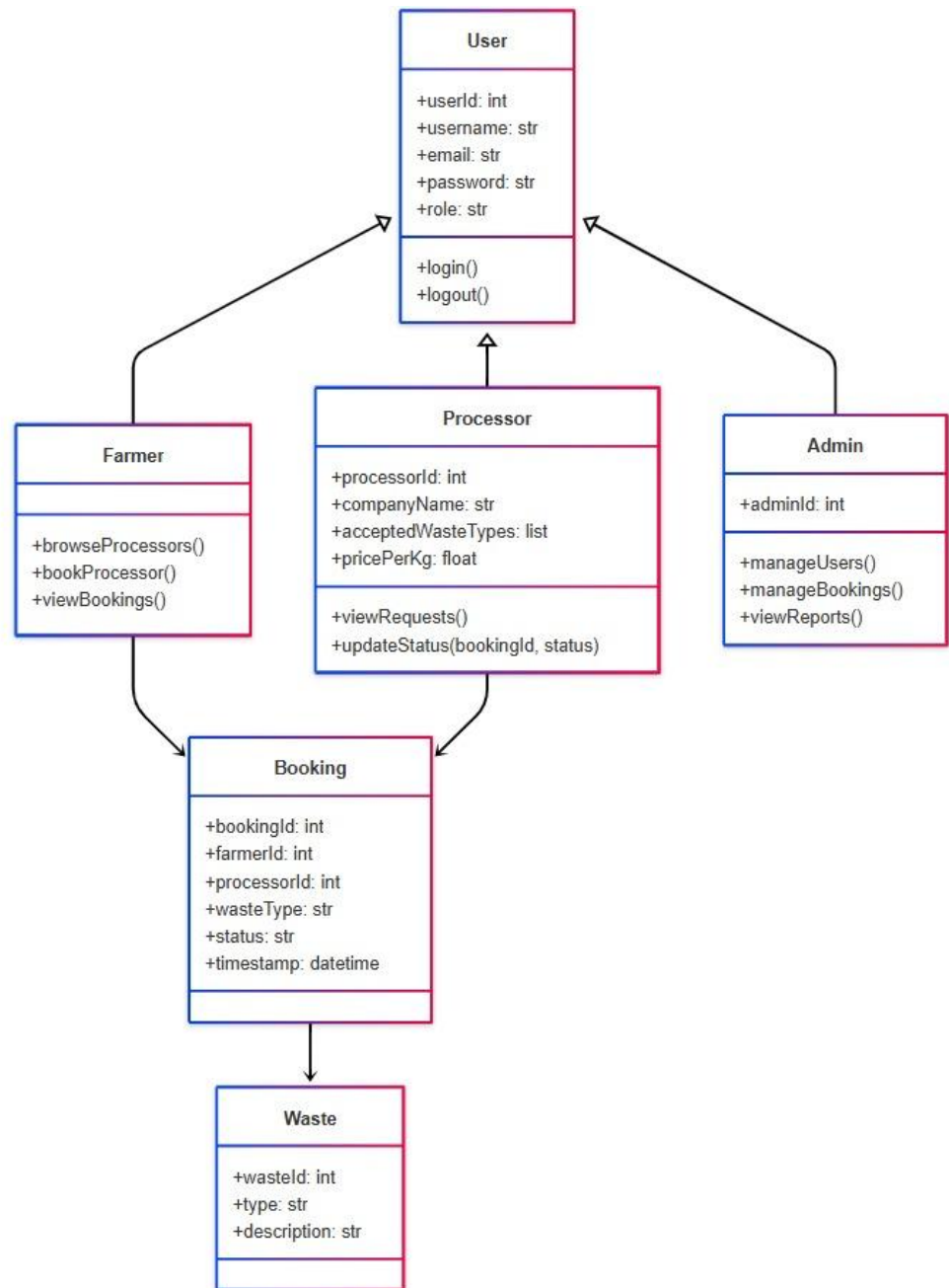


Fig 5.2 Class Diagram

5.2.3 Activity Diagram:

An activity diagram is a type of behavioral diagram in Unified Modeling Language (UML) used to represent the flow of activities within a system or process. It visualizes the sequence of actions, decisions, and control flows that occur as part of a specific process or workflow. In an activity diagram, activities are depicted as rounded rectangles, representing the tasks or actions to be performed. Arrows indicate the flow of control between activities, showing the sequence in which they are executed. Decision points are represented by diamond-shaped symbols, indicating branching paths or alternative courses of action based on conditions or decisions. Additionally, parallel activities can be depicted using fork and join symbols, representing concurrency in the process.

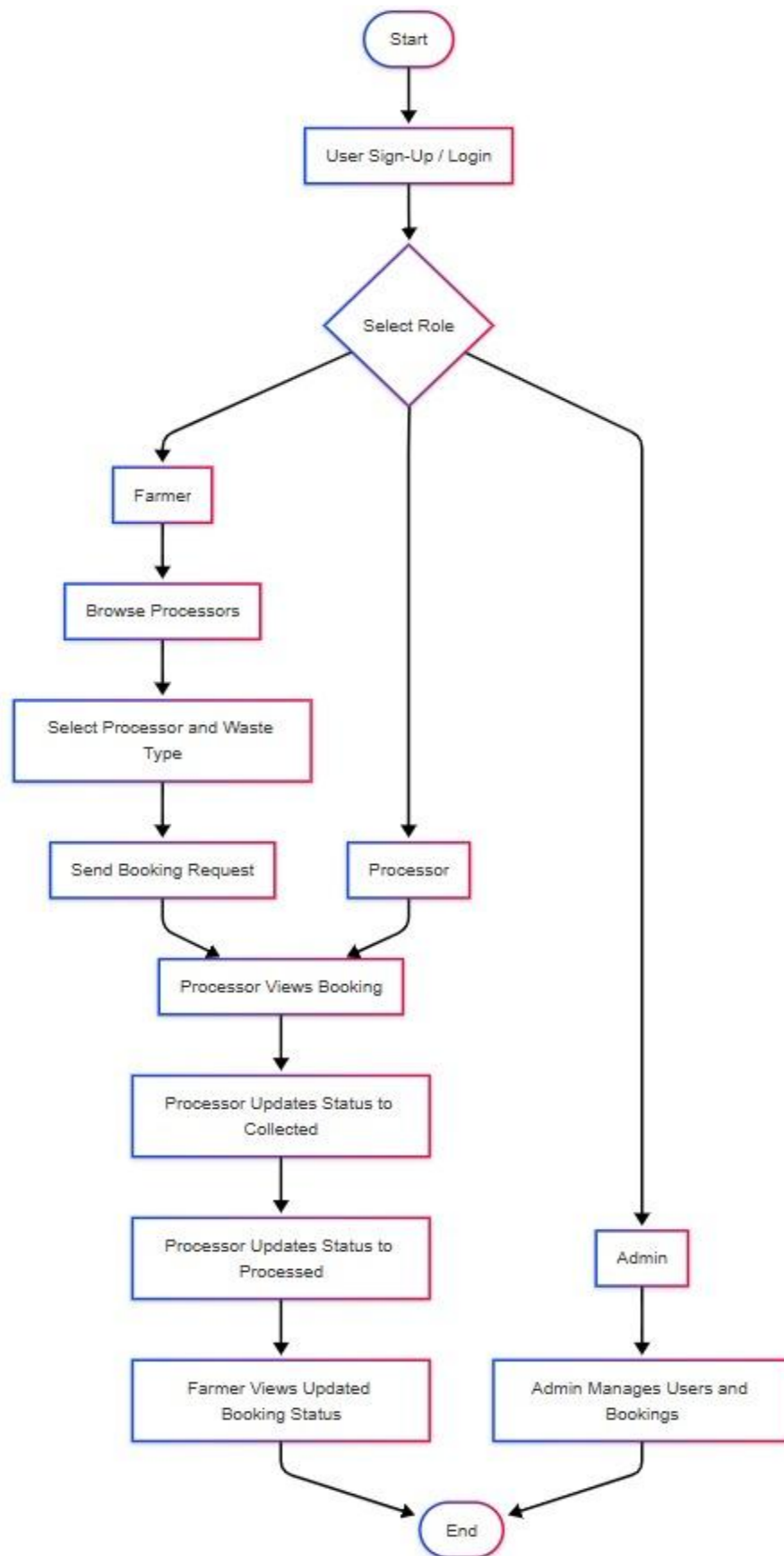


Fig 5.3 Activity diagram

5.2.4 Data flow diagram:

A data flow diagram (DFD) is a graphical representation of the flow of data within a system, illustrating how information moves between processes, data stores, and external entities. It consists of various symbols and components that depict the processes that manipulate the data, the data stores where information is held, the data flows that represent the movement of data between components, and the external entities that interact with the system.

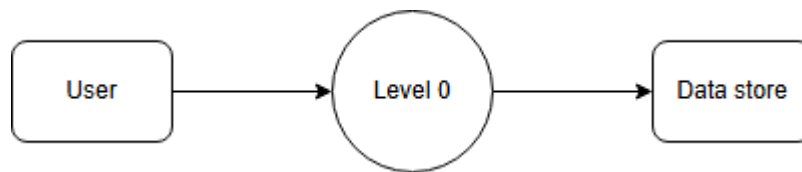


Fig 5.4 DFD Level 0

Fig 5.5 DFD Level 1

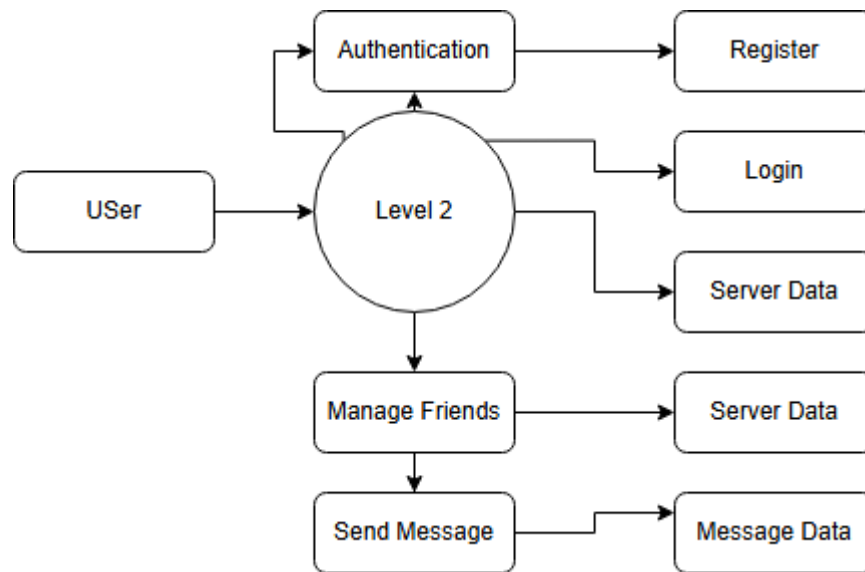
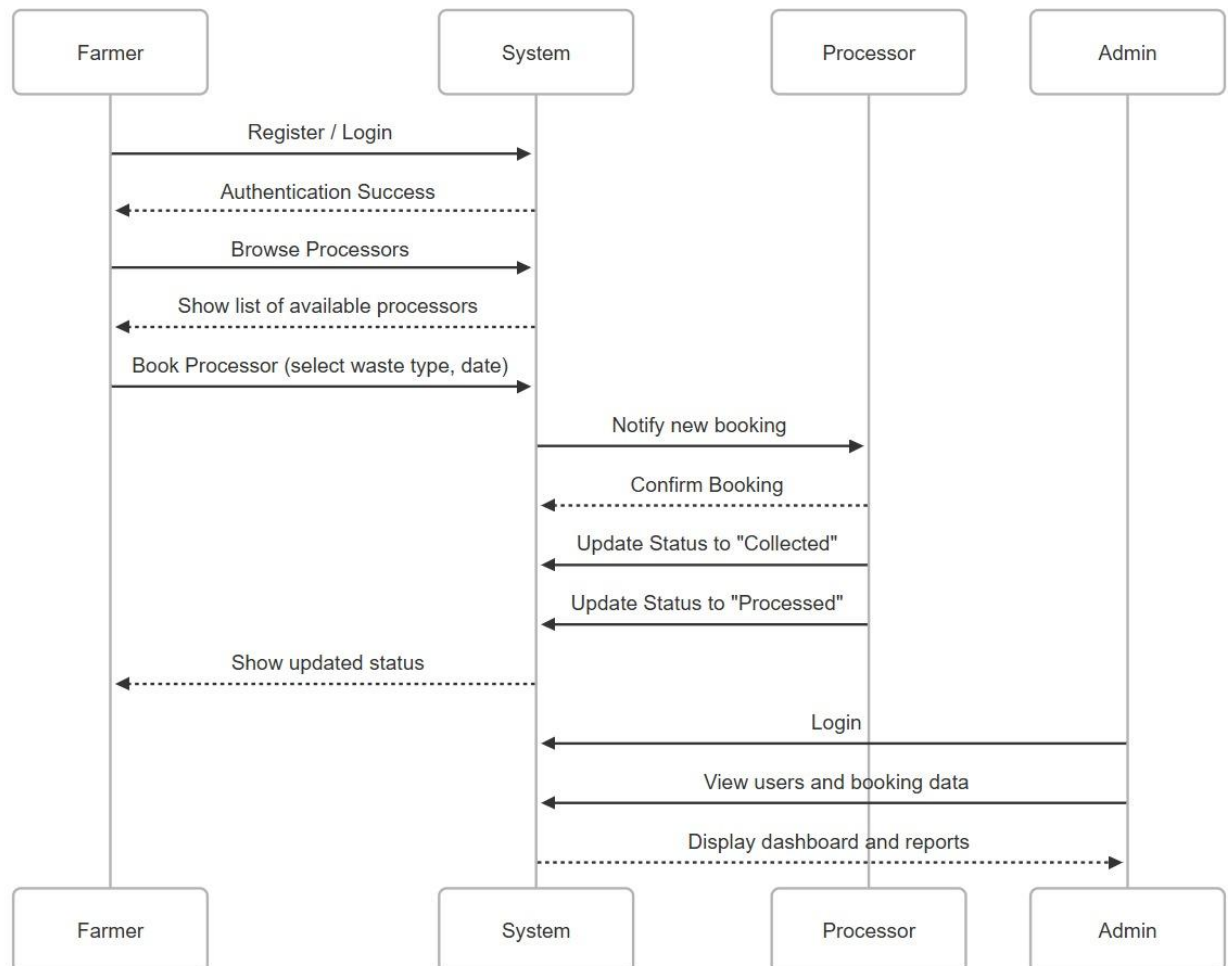


Fig 5.6 DFD Level 2

5.2.5 Sequence diagram:

A Sequence Diagram is a type of interaction diagram that depicts the interactions between objects or components within a system over time. It illustrates the flow of messages or method calls between different elements of the system, typically focusing on the sequence of interactions in a particular scenario or use case.



Sequence Diagram

5.2.6 ER-Diagram:

An Entity-Relationship (ER) diagram is a visual representation of the data model that illustrates the logical structure of a database. It is commonly used in database design to depict the relationships between different entities within a system.

In an ER diagram, entities are represented as rectangles, and relationships between entities are depicted using lines connecting them. Each entity is defined by its attributes, which are represented within the rectangle. Attributes describe the properties or characteristics of the entity.

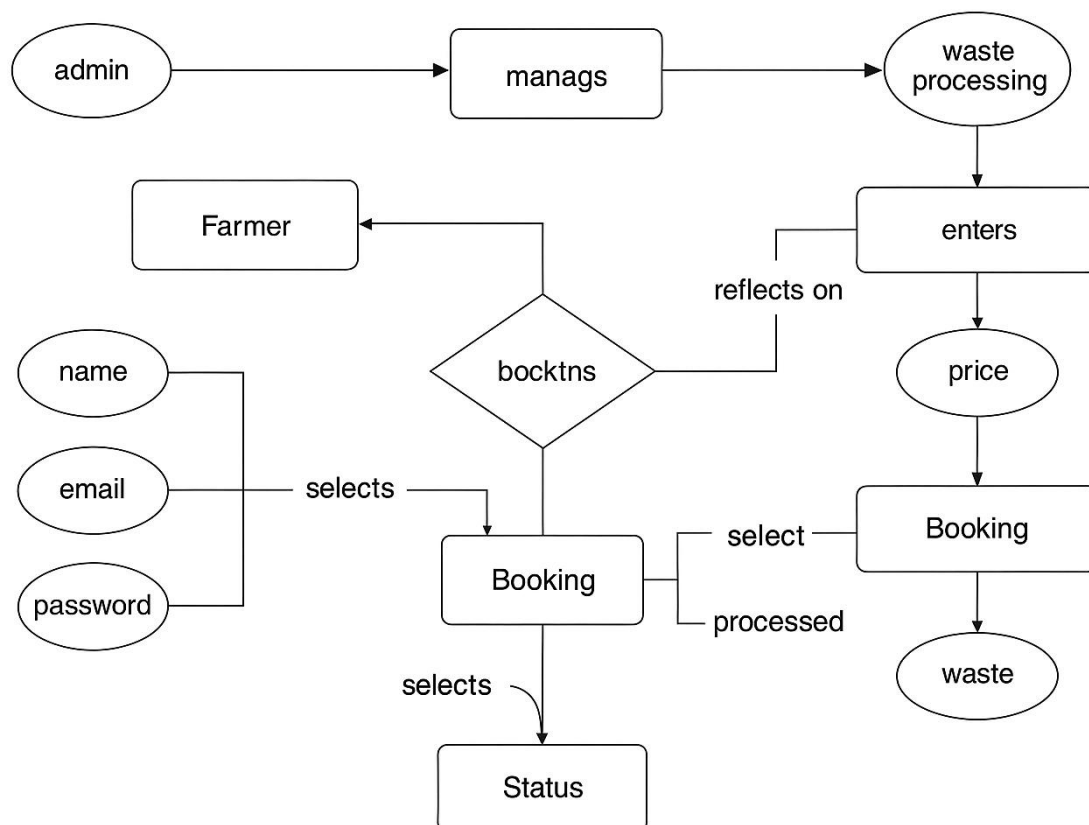


Fig 5.8 ER-Diagram

5.2.7 Data Dictionary:

A data dictionary is a centralized repository that provides detailed descriptions of data elements utilized within an organization's information systems. It functions as a comprehensive reference guide, offering metadata about data entities, attributes, definitions, relationships, and constraints. Each data element in the dictionary is defined with attributes such as name, description, data type, length, format, and constraints, providing a thorough understanding of its purpose, usage, and characteristics. With its structured documentation, the data dictionary serves as a vital resource for data management, database design, and system development, facilitating effective communication and standardization across various stakeholders involved in the organization's data processes.

1. User:

Field name	Data type	Constraint
User id	Number	Primary key
Username	String	Not null
Email	String	Unique,Not null
Password	String	Not null
Profile Image	String	Optional
Status	String	Default:offline

Table 5.1 User data dictionary

2. Server:

Field name	Data type	Constraint
Admin id	Number	Primary key
name	String	Not null
password	Number	Foreign key (Admin.Admin id)

Table 5.2 Server data dictionary

3.Appointment:

Field name	Data type	Constraint
Appointment_id	Number	Primary key
quantity	String	Not null
Waste_id	Number	Foreign key (User.User id)
Waste Price	Number	Foreign key(Channel.channel_id)
Time	String	Optional

Table 5.3 Message data dictionary

CHAPTER-6 IMPLEMENTATION

6.1 IMPLEMENTATION ENVIRONMENT:

- In this project, I have used Visual Studio Code as the primary development environment due to its robust support for modern web technologies and a wide range of useful extensions that enhance productivity. Package management and dependency handling are efficiently managed using npm, allowing for seamless installation and integration of required modules. For the front-end, I utilized Vite, a lightweight and high-performance bundler that supports hot module reloading, significantly improving development speed and workflow. The project is built using the MERN Stack—comprising MongoDB, Express.js, React.js, and Node.js—which provides excellent scalability, performance, and maintainability. MongoDB serves as the NoSQL database for storing essential data such as patient records, hospital details, vaccine inventories, and booking information. Express.js handles API routing and server-side logic, while Node.js powers the backend with a fast, event-driven architecture. On the front-end, React.js is used to create a dynamic and responsive user interface that adapts to different roles, including admin, hospital, and patient. Overall, this technology stack ensures efficient data handling, smooth integration between components, and rapid deployment of the VaxTrack vaccine management system.

6.2 MODULE SPECIFICATION:

Sign-Up/Login Form:

- Entry point for all users including **Admin**, **Farmer**, and **Processor** roles.
- New users can register with necessary details; returning users can log in securely.
- Role-based authentication ensures that users access only their relevant dashboard.

Processor Panel:

- Processors can register and enter their **waste processing details**, including the types of waste accepted and the prices offered for collection.
- They receive real-time booking requests from farmers.
- They can **update the booking status** to “Collected” and then “Processed” once the waste has been handled.

Farmer Panel:

- Farmers can browse through available processors based on **waste type** and **offered price**.
- They can book a processor for waste collection and monitor booking status.
- Provides a summary of past bookings and processor interaction history.

Admin Panel:

- Admins have full control over the platform.
- They can **view, verify, update, or delete** processor and farmer accounts.
- Monitor all bookings and system activity through an admin dashboard.
- Maintain platform integrity by handling invalid or incomplete data entries.

Booking and Status System:

- Farmers book processors through a simple form.
- Each booking reflects in the processor's panel instantly.
- Processors update status as "Collected" and then "Processed" for full tracking.

Communication and Notification Module:

- Users receive confirmation messages when a processor is booked.
- Notifications are sent to processors upon new bookings.
- Future scope includes integration with email or SMS for real-time alerts.

Automation & Future Integration (Optional):

- Potential to add **AI-based smart matching** between waste type and the most suitable processor.
- Future enhancements may include **WhatsApp reminders, chatbots** for support, and **geolocation-based processor recommendations**.

6.3 RESULTS

Home Page:

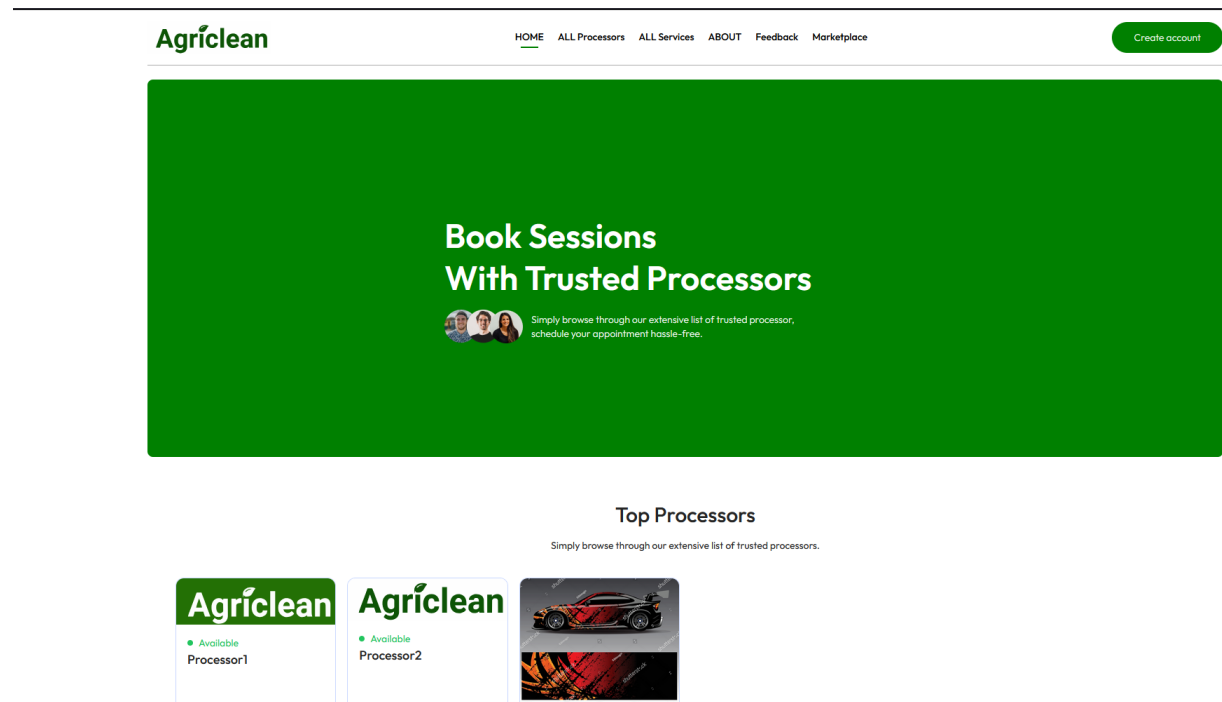


Fig 6.1 Home Page

Booking Page:

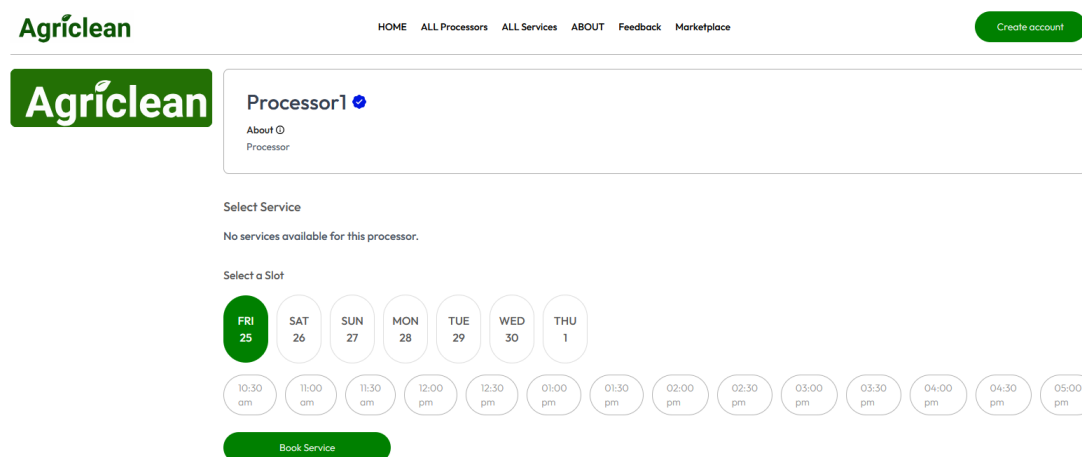


Fig 6.2 Booking Page

Marketplace:

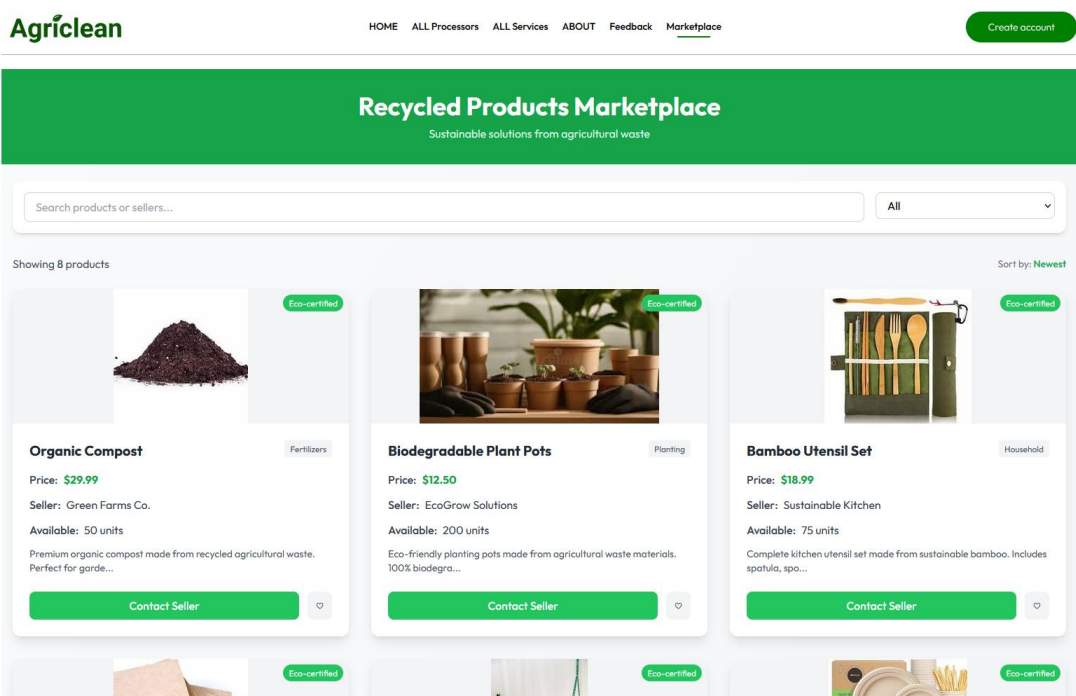


Fig 6.3 Marketplace

Create Account Page:

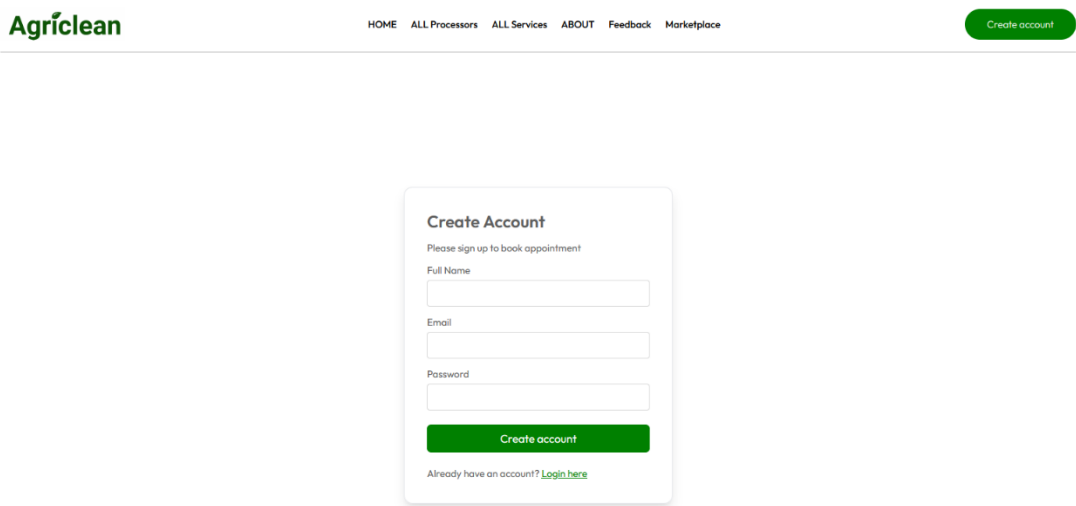


Fig 6.4 Create Account Page

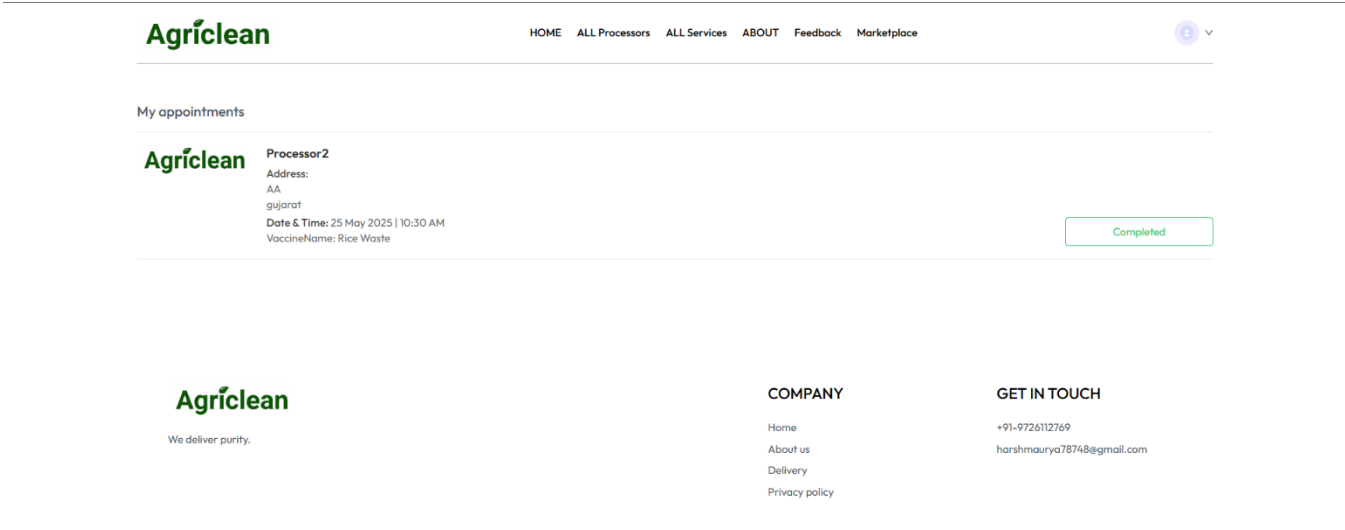


Fig 6.5 My Booking Page

Add Processors:

Add Processors

Upload Processor picture

Processor Name
Name

Processor Email
a@example.com

Set Password
.....

About Processor
Write about hospital

Address
Address 1
Address 2

Add Processor

Fig 6.6 Add Processor Page

Processor List:

All Processors

Agriclean
Processor1
Processor
☒ Available
☐ Approval

Agriclean
Processor2
Waste Processor
☒ Available
☐ Approval

Fig 6.7 All Processor

All bookings

Dashboard

Appointments

Add processor

ProcessorList

All Appointments

#	Farmer	Date & Time	Processor	Price	Action
1	 daxesh	25 May 2025, 10:30 AM	Agriclean	₹20*34	Completed
2	 Harsh	23 May 2025, 05:00 PM	Agriclean	₹20*30	Completed
3	 Harsh	23 May 2025, 03:00 PM	Agriclean	₹20*10	Completed

Fig 6.8 All Bookings

Edit Profile:

Agriclean

HOMEALL ProcessorsALL ServicesABOUTFeedbackMarketplace



daxesh

CONTACT INFORMATION

Email id:

harsh@gmail.com

Phone:

0000000000

Address:

BASIC INFORMATION

Gender:

Male

Birthday:

01-05-2004

Save information

Fig 6.9 Edit Profile

6.4 CODING STANDARD

Purpose of coding standards and best practices

To ensure the development of clean, maintainable, and scalable code in the Agriclean vaccine management system, coding standards and best practices were consistently followed. Variables and functions were named using camelCase, while file and folder names were kept lowercase with hyphens for clarity and consistency. React functional components with hooks were used for better performance and modularity. Backend APIs were structured using RESTful conventions, and logic was separated into routes, controllers, and middleware for clean architecture. Reusable components and utility functions were implemented to avoid code duplication. Proper indentation, meaningful comments, and consistent formatting were maintained using tools like ESLint and Prettier. These practices made the codebase easy to understand, test, and extend by any developer involved in the project.

CHAPTER-7 TESTING

7.1 TEST PLANNING

A test plan is a crucial element of any successful software testing process. It outlines the overall testing approach, defines the scope and objectives, and serves as a comprehensive summary of all testing activities. A well-structured test plan helps the team understand how tests will be executed, what tools and environments are required, and what criteria define a successful outcome.

The primary goal of test planning is to ensure that all functional and non-functional requirements are thoroughly verified. This involves listing all testable features, defining test cases, assigning responsibilities, and setting timelines. A strong test plan helps in identifying potential issues early and ensures that testing efforts align with project goals.

During the early stages of Agriclean's development, the test strategy was clearly defined, including methodologies such as manual testing, black-box, and white-box testing. Hardware and software environments were taken into account, ensuring compatibility across major browsers and devices. Testing focused on critical features such as user registration, waste processor booking, status updates, and role-based access across Admin, Farmer, and Processor panels.

A robust test plan also anticipated risks like delayed responses, UI misalignments, or failed booking updates. By thoroughly mapping the testing activities, tool usage, and validation goals, the plan ensured that the Agriclean platform was well-tested, functionally complete, and ready for deployment.

In testing there are some steps to followed for testing a website:

- **Functionality Testing:**

Verified that all clickable elements such as buttons, links, forms, logins, and dashboards on each role-specific panel worked correctly. Input validation and error messaging were tested for accuracy and edge cases.

- **Compatibility Testing:**

Checked that the system was fully responsive and functional across major browsers including Chrome, Firefox, and Edge, as well as on different screen sizes like desktop, tablets, and smartphones.

- **Usability Testing:**

Focused on UI clarity and logical navigation. Tests were conducted from the user perspective (farmer, processor, admin) to ensure intuitive flows for actions like booking, updating status, or managing users.

- **Performance Testing:**

Tools like Google PageSpeed Insights were used to test loading speed and responsiveness. Simulated multiple simultaneous bookings to test the system's ability to handle concurrent usage and ensure backend optimization.

7.2 TESTING METHODS

There are mainly two strategies:

Black box testing

- Black Box Testing is a software testing method that focuses on evaluating the functionality of an application without any knowledge of its internal code structure, implementation details, or internal paths. In this approach, the tester is concerned only with inputs and expected outputs, treating the system as a "black box." The internal logic of the program remains unknown to the tester. This technique is ideal for validating how the system behaves under various input conditions, verifying whether it meets the specified requirements and user expectations.
- **Black Box Testing** is a software testing method that focuses on evaluating the functionality of an application without having any knowledge of its internal code structure, implementation details, or logical paths. In this approach, the tester interacts with the application by providing inputs and observing the outputs, without any concern for the internal workings of the system. This technique is particularly useful for verifying whether the system behaves as expected under various scenarios and meets all defined user requirements.
- In the **Agriclean** waste management system, black box testing was applied to all user-facing modules including the **Farmer Panel**, **Processor Panel**, and **Admin Panel**. For instance, login and registration forms were tested with both valid and

invalid inputs to ensure proper validation, error messages, and field behaviors. On the farmer side, the waste booking form was tested to confirm that processor selection and waste type inputs correctly triggered bookings. On the processor side, status updates like "Collected" and "Processed" were checked to ensure the user interface responded correctly and reflected the changes in real time.

- This testing method helped identify issues such as form input validation failures, missing or incorrect UI messages, inconsistent navigation between dashboards, and improper error handling during incorrect actions. Since the testers did not interact with the underlying logic or database, the tests closely replicated how real users would engage with the platform. As a result, black box testing was highly effective in uncovering functional and usability flaws, ensuring that the user experience was intuitive and reliable across all roles in the Agriclean platform.

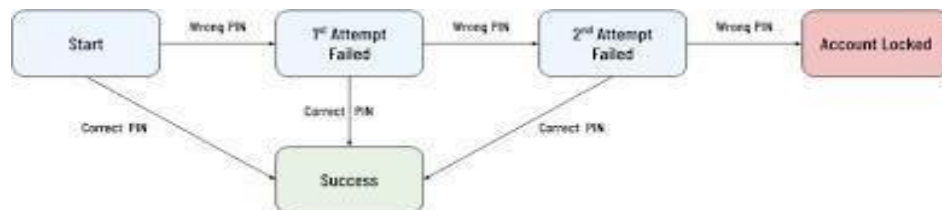


Fig 7.1 Blackbox testing

White Box Testing

- White Box Testing, also known as structural or clear-box testing, involves inspecting the internal logic and structure of the application. This includes testing the backend functions, loops, conditions, data flow, and control logic.
- In Agriclean, white box testing was used extensively to validate backend APIs, authentication logic, and route responses. Special focus was given to validating booking status updates, data storage for processor details, and role-based access control. Each route was tested using tools like Postman to ensure consistent and secure data handling. Additionally, unit tests were written for utility functions to ensure robust error handling. This type of testing was crucial in identifying hidden bugs, optimizing backend logic, and ensuring a stable and secure codebase.
- Both black box and white box testing played essential roles in validating Agriclean. While black box testing ensured real-world usability, white box testing ensured that internal operations were bug-free and optimized for performance.

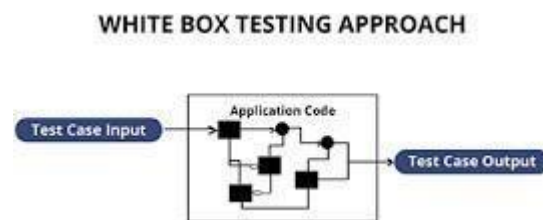


Fig 7.2 Whitebox testing

7.3 TEST CASES

Sr no.	Test Cases	Status
1.	Click through all pages and verify each feature (Home, Booking, Dashboard)	Pass
2.	Test all forms, input fields, buttons, and links for correct functionality	Pass
3.	Test on Chrome, Firefox, Edge, and mobile browsers for compatibility	Pass
4.	Validate navigation flow and UI consistency across all panels	Pass
5.	Book waste collection and verify processor update flow	Pass
6.	Simulate multiple users to test system performance and backend handling	Pass

Table 7.1 testing result

CHAPTER-8 CONCLUSION AND DISCUSSION

8.1 OVERALL ANALYSIS OF INTERNSHIP / PROJECT VIABILITIES

- The development of the Agriclean waste management system during the internship provided a valuable opportunity to implement theoretical knowledge in a real-world project. This application encompassed the full software development lifecycle—beginning with requirement analysis, UI/UX design, and progressing through development, testing, and deployment—using modern web technologies such as the MERN stack and RESTful APIs.
- The concept of Agriclean addresses real-life challenges in agricultural waste management, including lack of coordination between farmers and processors, inefficient communication, and absence of centralized data. By introducing a structured, role-based system for Admin, Farmer, and Processor, the platform proved both relevant and practically viable for implementation in rural and agri-industrial settings.
- The system effectively streamlined user interactions, providing real-time updates on waste bookings and status changes, which enhanced overall transparency and traceability in the waste collection and processing cycle.
- From a technical perspective, the project utilized modular coding, scalable data models, and a responsive interface to ensure adaptability and performance. Security and role-based access control were also considered in the system design. Features like booking tracking, processor pricing, and status updates added to the application's usability and scope.
- This internship not only enhanced my full-stack development skills but also offered hands-on experience with collaborative development, agile planning, and system architecture design. It was a valuable opportunity to apply software engineering principles to create a socially beneficial and scalable digital solution.

8.2 SUMMERY OF INTERNSHIP

- During my 12-week internship at Grownited Private Limited, I developed the Agriclean waste management platform, which significantly enhanced my understanding of full-stack web development. I was involved in all stages of the development process, including wireframe design, database modeling, frontend and backend integration, API creation, and thorough testing. Through hands-on experience with technologies like React.js, Node.js, Express.js, and MongoDB, I gained practical knowledge in session handling, user authentication, and data validation. Working closely with mentors and team members not only improved my technical proficiency but also strengthened my problem-solving, communication, and project management skills, preparing me for future challenges in the software industry.



8.3 LIMITATION AND FUTURE ENHANCEMENT

8.3.1 Limitation

- Currently this project is not properly optimized
- Limited geolocation accuracy and no integration with advanced map services.

8.3.2 Future enhancement

- In the future, an AI-powered chatbot can be integrated to assist users with vaccine-related queries and booking support. It can also help assess infection risks based on symptoms and provide personalized health tips. This enhancement will improve user experience and offer 24/7 virtual healthcare guidance.

8.4 CONCLUSION

- The **Agriclean** waste management system is a web-based solution designed to streamline the process of agricultural waste collection and processing by connecting farmers, processors, and administrators on a single platform. It enables processors to register and list their services and prices, while allowing farmers to book processors based on waste type and availability. Real-time updates ensure that booking statuses can be tracked as “Collected” and “Processed,” providing transparency and accountability. Admins manage user data and oversee all operations, ensuring the system runs smoothly. With its role-based design, intuitive interface, and scalable architecture, Agriclean offers an efficient, modern, and environmentally conscious approach to agricultural waste management.

CHAPTER-9 REFERENCE

1. React documentation: <https://react.dev/>
2. MDN JavaScript documentation: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
3. MDN CSS docs: <https://developer.mozilla.org/en-US/docs/Web/CSS>
4. MDN html docs: <https://developer.mozilla.org/en-US/docs/Web/HTML>
5. MongoDB Documentation – <https://www.mongodb.com/docs>
6. Express.js Guide – <https://expressjs.com/>
7. React Official Documentation – <https://reactjs.org/docs/getting-started.html>
8. Node.js Official Docs – <https://nodejs.org/en/docs>
9. JWT (JSON Web Token) – <https://jwt.io/introduction/>
10. Mongoose ODM Documentation – <https://mongoosejs.com/docs/guide.html>
11. Tailwind CSS Documentation – <https://tailwindcss.com/docs>
12. Chart.js Documentation – <https://www.chartjs.org/docs/latest/>
13. Postman API Tool – <https://www.postman.com/>
14. GitHub (Project Hosting) – <https://github.com/>
15. Razorpay / Stripe (Payment Integration Reference) – <https://razorpay.com/docs/> / <https://stripe.com/docs>
16. W3Schools Web Tutorials – <https://www.w3schools.com/>
17. MDN Web Docs – <https://developer.mozilla.org/>
18. Stack Overflow – <https://stackoverflow.com/>
19. Figma (UI Design Tool) – <https://www.figma.com/>